

UNIDAD 2.1

Introducción a React

Componentes, JSX y ecosistema



React

Vite

JSX

Componentes

AGENDA

Lo que veremos en este tema

En este contenido

01 Ventajas de React

02 Configuración de un proyecto

03 Estructura de la aplicación

04 Creación de un componente

Más adelante en la unidad

05 Uso de props

06 Uso del estado

07 Uso de eventos

08 Herramientas de desarrollo

ORÍGENES

De una necesidad de Facebook a un estándar de la industria

React nace como la respuesta de Meta para ayudar a más personas a trabajar en el código base de Facebook y ofrecer nuevas funciones con mayor rapidez.

Funcionó tan bien que Meta lo convirtió en código abierto. Hoy es una biblioteca madura para crear interfaces de usuario basadas en componentes, con una comunidad y un ecosistema enormes.



Creada por Meta

Mantiene Facebook en producción



Código abierto

Gratuita y con enorme comunidad



Netflix · Uber · Airbnb

Entre muchas otras empresas

VENTAJAS

¿Por qué React es tan popular?



Sencillez

Se centra en hacer una cosa muy bien: proporcionar un mecanismo potente para crear componentes de interfaz de usuario.



Componentes reutilizables

Piezas de interfaz que se combinan para crear un frontend, y que se pueden reutilizar en distintas pantallas o aplicaciones.



Adopción incremental

Puede incorporarse a una app existente, aunque use otro framework, porque no necesita hacerse cargo de toda la aplicación.



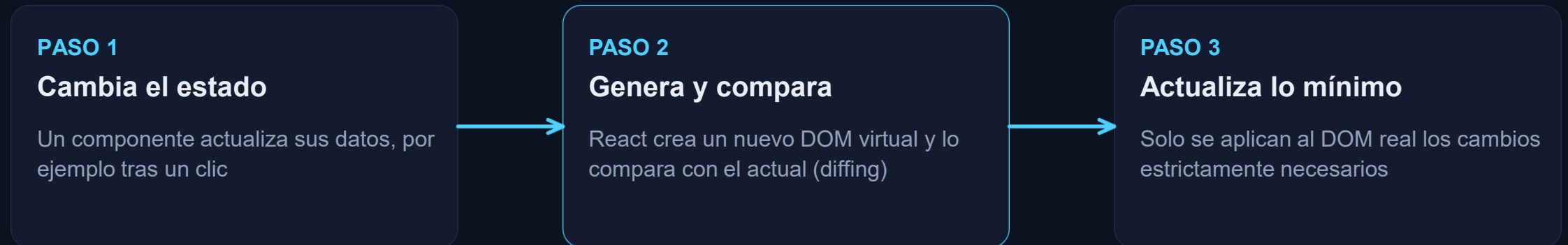
Ecosistema y aprendizaje

Comunidad enorme, muchos recursos de aprendizaje y herramientas como Vite que agilizan la creación de proyectos.

RENDIMIENTO

El DOM virtual

Los cambios en el DOM real son costosos. React usa una representación en memoria, el DOM virtual, para calcular el mínimo cambio necesario.



Un ecosistema que sigue creciendo



Componentes de servidor

Permiten una recuperación de datos de alto rendimiento y reducen la cantidad de JavaScript enviada al navegador.



React Native

Un framework basado en React para crear apps multiplataforma en iOS y Android, reutilizando habilidades y código.



Calidad garantizada

Meta usa React en Facebook, así que cuida su calidad y facilita que las nuevas versiones sean baratas de adoptar.

ANTES DE EMPEZAR

Herramientas que vamos a usar



Navegador

Un navegador moderno como Google Chrome



Terminal

Para ejecutar comandos y crear el proyecto React



Visual Studio Code

code.visualstudio.com



Node.js y npm

nodejs.org/en/download

CONFIGURACIÓN

Creación de un proyecto con Vite

- 1 Ejecuta el comando y escribe el nombre del proyecto
- 2 Selecciona el framework **React**
- 3 Selecciona la variante **JavaScript**

```
$ npm create vite@latest
```

```
✓ Project name: ... mi-app-react
```

```
✓ Select a framework: > React
```

```
✓ Select a variant: > JavaScript
```

```
$ cd mi-app-react
```

```
$ npm install
```

```
$ npm run dev
```

Carpetas y archivos generados

Archivo / carpeta	Descripción
node_modules	Paquetes npm de los que depende el proyecto
public	Recursos estáticos servidos en la ruta raíz /
src/main.jsx	Punto de entrada: carga la app React en index.html
src/App.jsx	Componente React de nivel superior, llamado App
eslint.config.js	Configuración de ESLint para revisar el código
index.html	Página web raíz donde se carga la aplicación React
package.json	Nombre, versión, dependencias y scripts del proyecto

CALIDAD DE CÓDIGO

Linting con ESLint

El **linting** revisa el código en busca de posibles problemas a medida que se escribe. Vite ya instala y configura ESLint por nosotros.

- ✓ Instala la extensión ESLint en VS Code
- ✓ Activa `eslint.probe` para JS y JSX
- ✓ Desactivamos la regla de `prop-types`

```
// eslint.config.js
export default [
  ...,
  {
    rules: {
      ...,
      'react/prop-types': 'off',
    },
  },
];
```

CALIDAD DE CÓDIGO

Formateo con Prettier

Prettier formatea el código automáticamente para mantener un estilo coherente. Vite no lo configura, así que lo instalamos aparte.

```
$ npm i -D prettier
$ npm i -D eslint-config-prettier
```

Activa **Formatear al guardar** y elige Prettier como formateador predeterminado en VS Code.

```
// .prettierrc.json
{
  "printWidth": 100,
  "singleQuote": true,
  "semi": true,
  "tabWidth": 2,
  "trailingComma": "all",
  "endOfLine": "auto"
}
```

Modo desarrollo y compilación de producción

▶ Desarrollo

```
npm run dev
```

Servidor de desarrollo de Vite en localhost:5173

Transpila JSX a JavaScript al vuelo

Recarga la app automáticamente al guardar cambios

⚡ Producción

```
npm run build
```

Minificación: elimina espacios y acorta nombres

Empaquetado: fusiona archivos en paquetes (bundles)

Tree-shaking: elimina el código no utilizado

Genera los artefactos en la carpeta dist

ESTRUCTURA DE LA APP

El punto de entrada: main.jsx

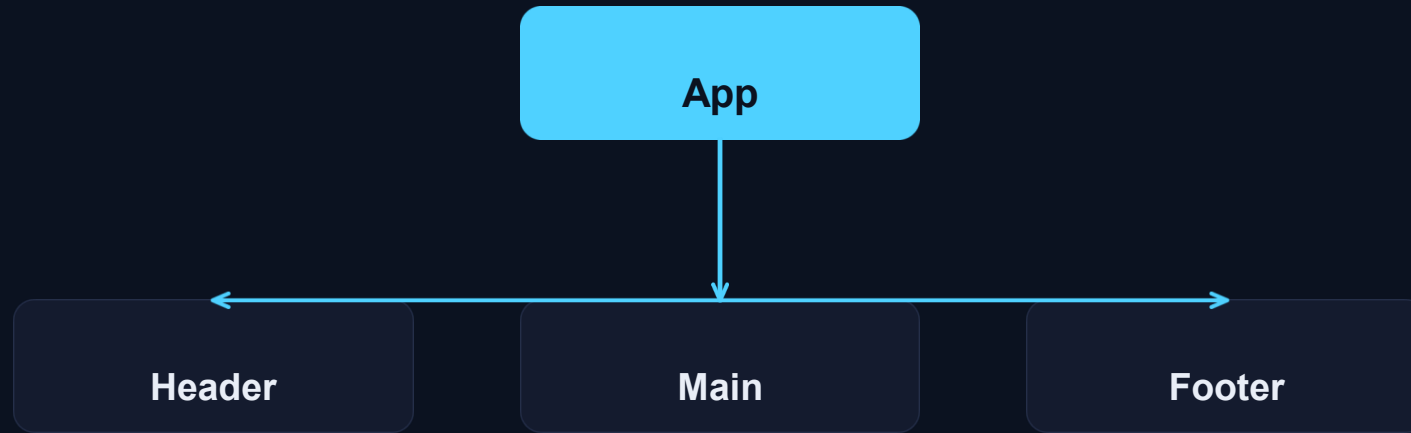
- **createRoot** crea una raíz en el elemento con id "root" del HTML
- **render** muestra los componentes React indicados (renderizado)
- **StrictMode** activa comprobaciones extra en desarrollo

```
// src/main.jsx
createRoot(document.getElementById('root')).
render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

JSX

JSX y el árbol de componentes

JSX (JavaScript XML) permite escribir código similar a HTML dentro de JavaScript. Debe transpilarse a llamadas de función normales antes de ejecutarse.



COMPONENTES

Anatomía de un componente

Un componente React es simplemente una **función JavaScript** normal que devuelve JSX

El **fragmento** `<></>` agrupa elementos sin crear un nodo extra en el DOM

Las **llaves** `{ }` permiten insertar JavaScript dentro del JSX

```
// src/App.jsx
function App() {
  return (
    <>
      <button onClick={() =>
        setCount((count) => count + 1)}>
        count es {count}
      </button>
    </>
  );
}
```

MANOS A LA OBRA

Creando el componente Alert

Un componente que muestra un icono, un título y un mensaje. Se crea en un nuevo archivo **Alert.jsx** dentro de src.

⚠ El nombre de un componente React debe empezar con **mayúscula**; si no, React lo trata como un elemento HTML.

```
// src/Alert.jsx
function Alert() {
  return (
    <div>
      <div>
        <span role="img" aria-label="Advertencia">⚠</span>
        <span>¡Oh, no!</span>
      </div>
      <div>Ha ocurrido un error</div>
    </div>
  );
}
```

SINTAXIS ALTERNATIVA

Función normal o función flecha

No hay diferencias significativas entre ambas para un componente de función. La elección depende de tu preferencia personal.

Función normal

```
function Alert() {  
  return (  
    ...  
  );  
}
```

Función flecha

```
const Alert = () => {  
  return (  
    ...  
  );  
};
```

REUTILIZAR COMPONENTES

Exportar e importar

Cada componente suele vivir en su propio archivo, así que debe **exportarse** antes de poder usarse en otro componente.

Esto también resuelve el aviso de ESLint sobre un componente que no se utiliza.

```
// src/Alert.jsx - 1. exportar
export function Alert() {
  ...
}
```

```
// src/App.jsx - 2. importar
import { Alert } from './Alert';
```

RESULTADO

Usar Alert dentro de App

- 1 Reemplaza el contenido de App para que solo renderice Alert
- 2 Ejecuta **npm run dev** y abre la app en el navegador
- 3 El componente Alert ya forma parte del árbol y se muestra en pantalla

```
// src/App.jsx  
function App() {  
  return <Alert />;  
}
```